

Exp No: 8

GAS SENSOR MQ-2 IN PROTEUS USING ARDUINO

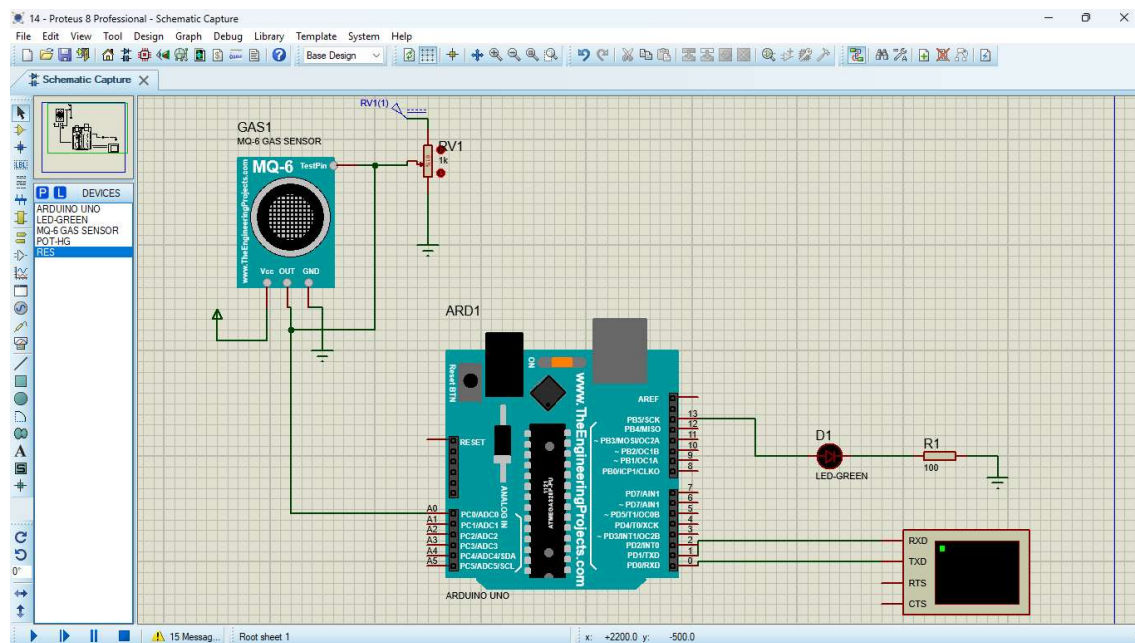
AIM:

To write an Embedded C program for interfacing Gas Sensor MQ-2 using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:



PROGRAM:

```
#define GreenLed 13
```

```
#define Sensor A0
```

```
void setup(){  
    pinMode(13,OUTPUT);  
    pinMode(12,OUTPUT);  
    pinMode(A0,INPUT);  
    Serial.begin(9600);
```

```

}

void loop() {
  int value = analogRead(A0);
  Serial.print("Analogic Value coming from the sensor : ");
  Serial.println(value);
  delay(100);
  if(value>600){
    digitalWrite(13,HIGH);
  }
  else{
    digitalWrite(13,LOW);
  }
  delay(20);
}

```

Code for MQ6 sensor:

```
int mq6Pin = A0; // MQ-5 analog pin connected to A0
```

```

void setup() {
  Serial.begin(9600);
}

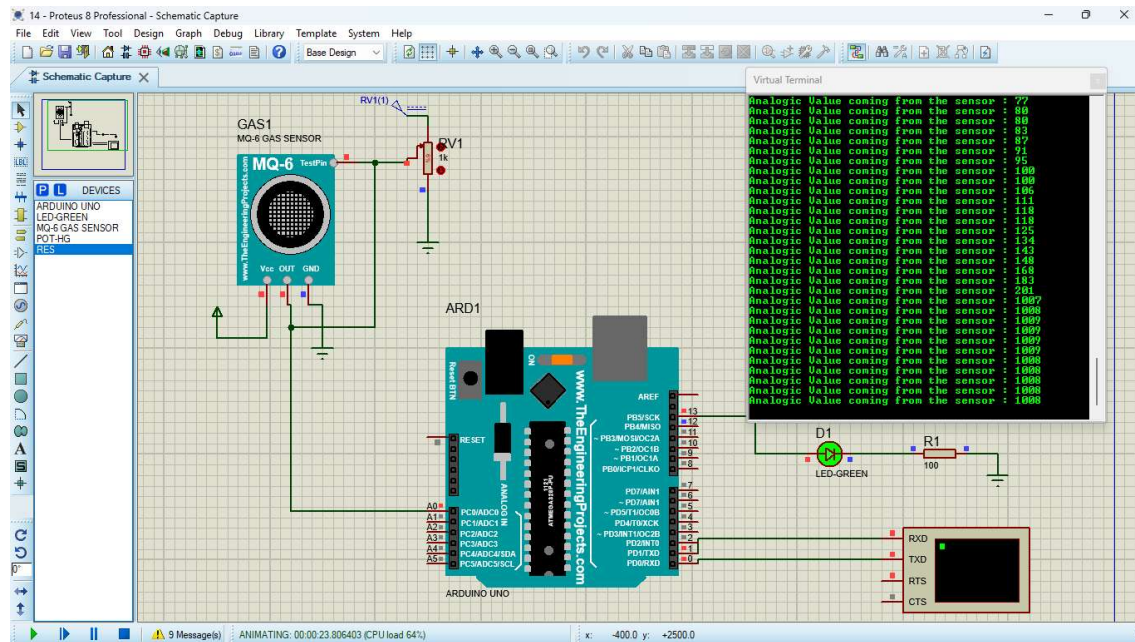
```

```

void loop() {
  int sensorValue = analogRead(mq5Pin);
  Serial.print("MQ5 Sensor Value: ");
  Serial.println(sensorValue);
  delay(1000); // 1-second delay
}

```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 9

ULTRASONIC SENSOR IN PROTEUS USING ARDUINO

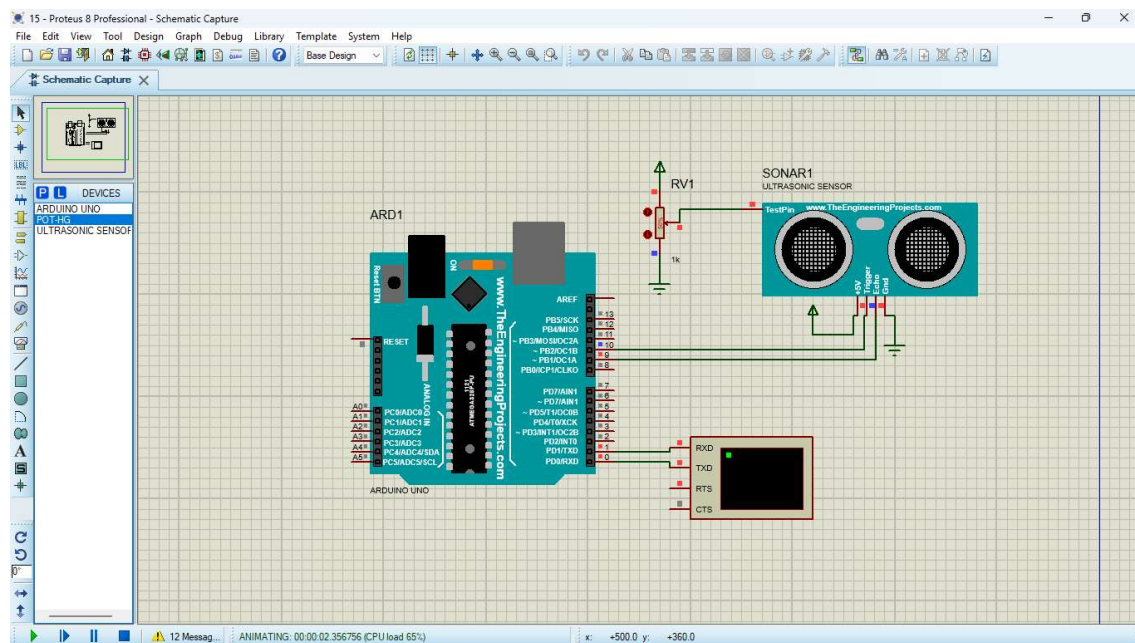
AIM:

To write an Embedded C program for interfacing Ultrasonic Sensor using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:



PROGRAM:

// Program Sketch

```
int trig = 10;
```

```
int echo = 9;
```

```
long duration;
```

```
int cm;
```

```
void setup() {
```

```
    // put your setup code here, to run once:
```

```
    pinMode(trig, OUTPUT);
```

```
    pinMode(echo, INPUT);
```

```

Serial.begin(9600);

}

void loop() { // Sketch to calculate ultrasonic distance

    digitalWrite(trig, LOW);

    delayMicroseconds(10);

    digitalWrite(trig, HIGH);

    delayMicroseconds(10);

    digitalWrite(trig, LOW);

    delayMicroseconds(10);

    duration = pulseIn(echo, HIGH);

    cm = (duration/2) * 0.034;

    // Print Value on Serial Monitor

    Serial.print(" Distance = ");

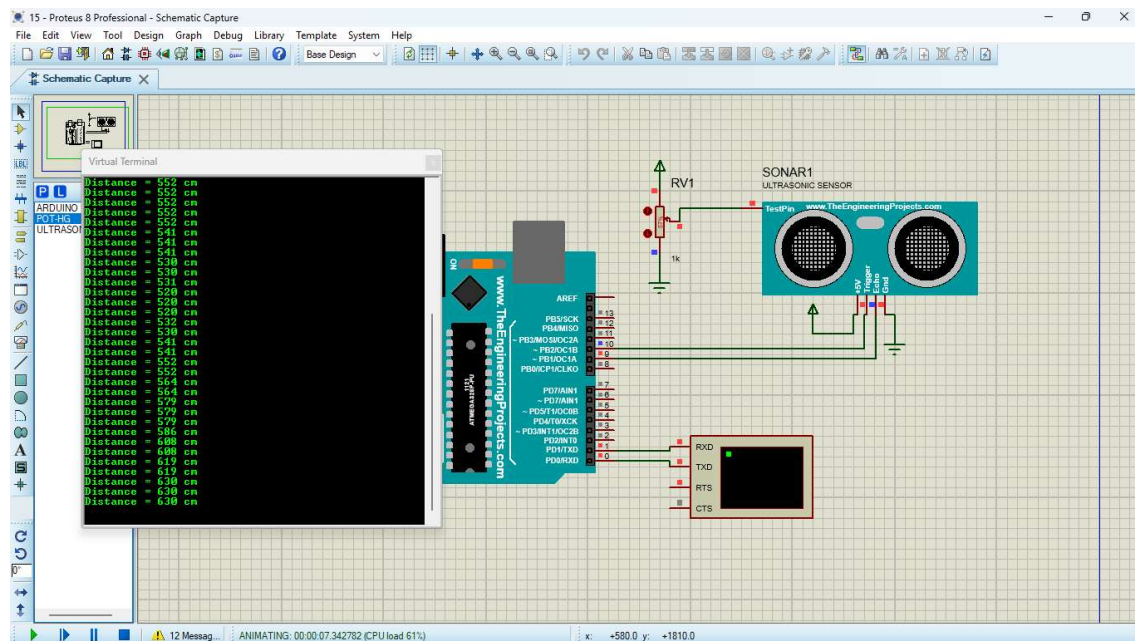
    Serial.print(cm);

    Serial.println(" cm");

}

```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 10

PIR SENSOR IN PROTEUS USING ARDUINO

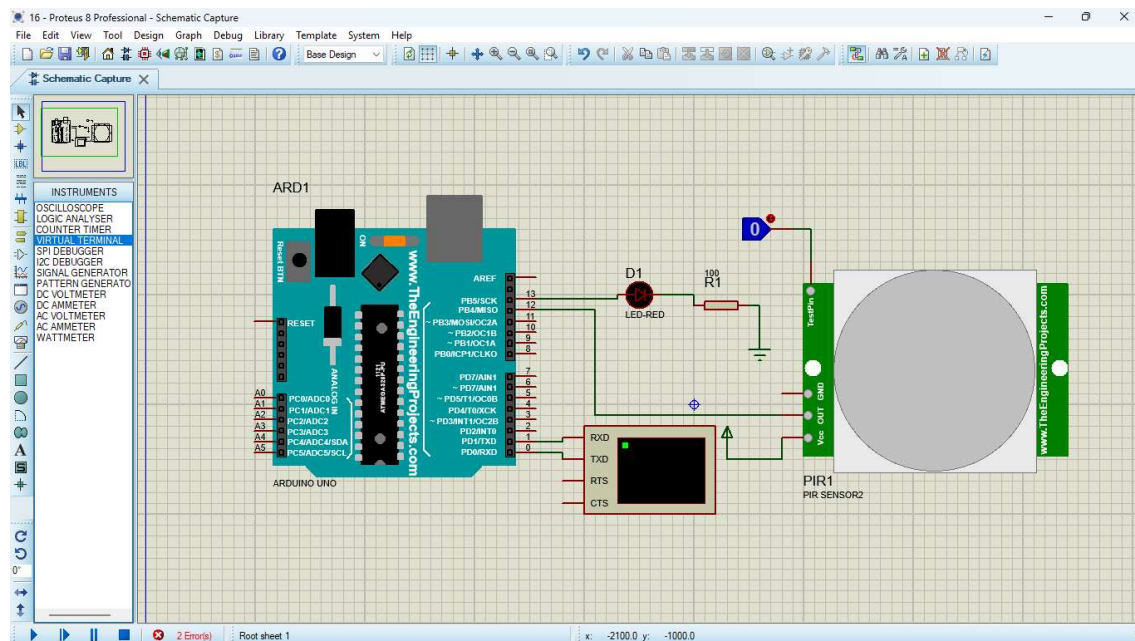
AIM:

To write an Embedded C program for interfacing PIR Sensor using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:



PROGRAM:

```
int Status = 13; // Digital pin D6

int sensor = 12; // Digital pin D7

void setup() {

    Serial.begin(9600);

    pinMode(sensor, INPUT); // declare sensor as input

    pinMode(Status, OUTPUT); // declare LED as output

}
```

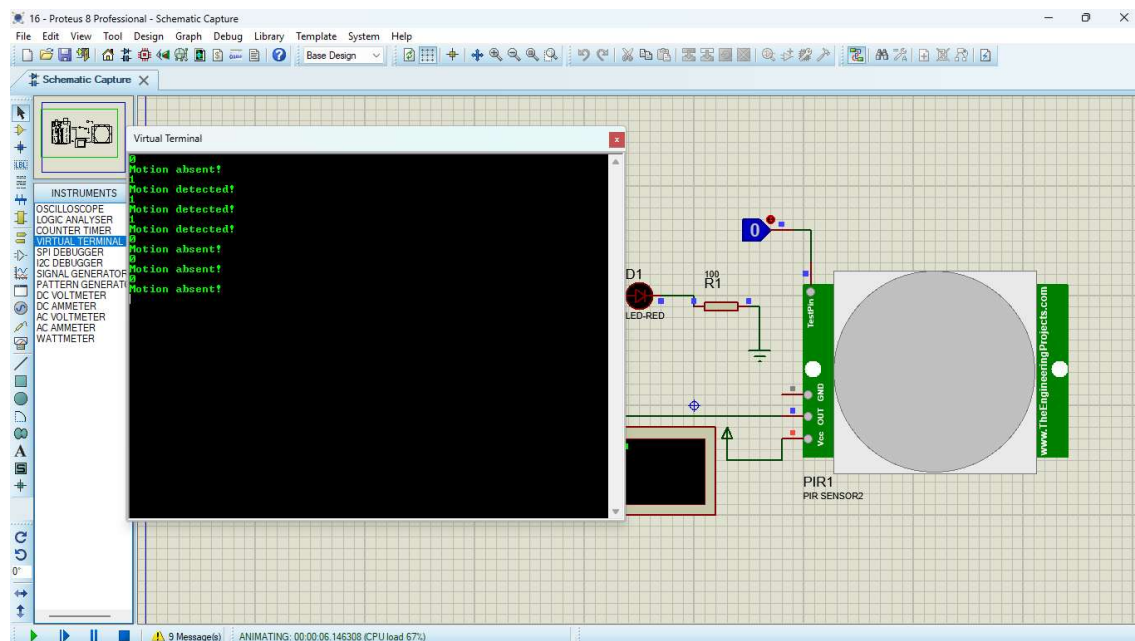


```

void loop() {
    long state = digitalRead(sensor);
    Serial.println(state);
    if(state == HIGH) {
        digitalWrite (Status, HIGH);
        Serial.println("Motion detected!");
        delay(1000);
    }
    else {
        digitalWrite (Status, LOW);
        Serial.println("Motion absent!");
        delay(1000);
    }
}

```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 11

STUDY OF GPIO & BIT ADDRESSING IN AT89C51

AIM:

To study GPIO control and bit-addressable SFR using Embedded C.

SOFTWARE REQUIRED:

- Keil

THEORY

8051 ports are byte and bit addressable. Using sbit, individual bits of a port can be accessed directly. Ports default to logic HIGH due to internal pull-ups.

PROGRAM:

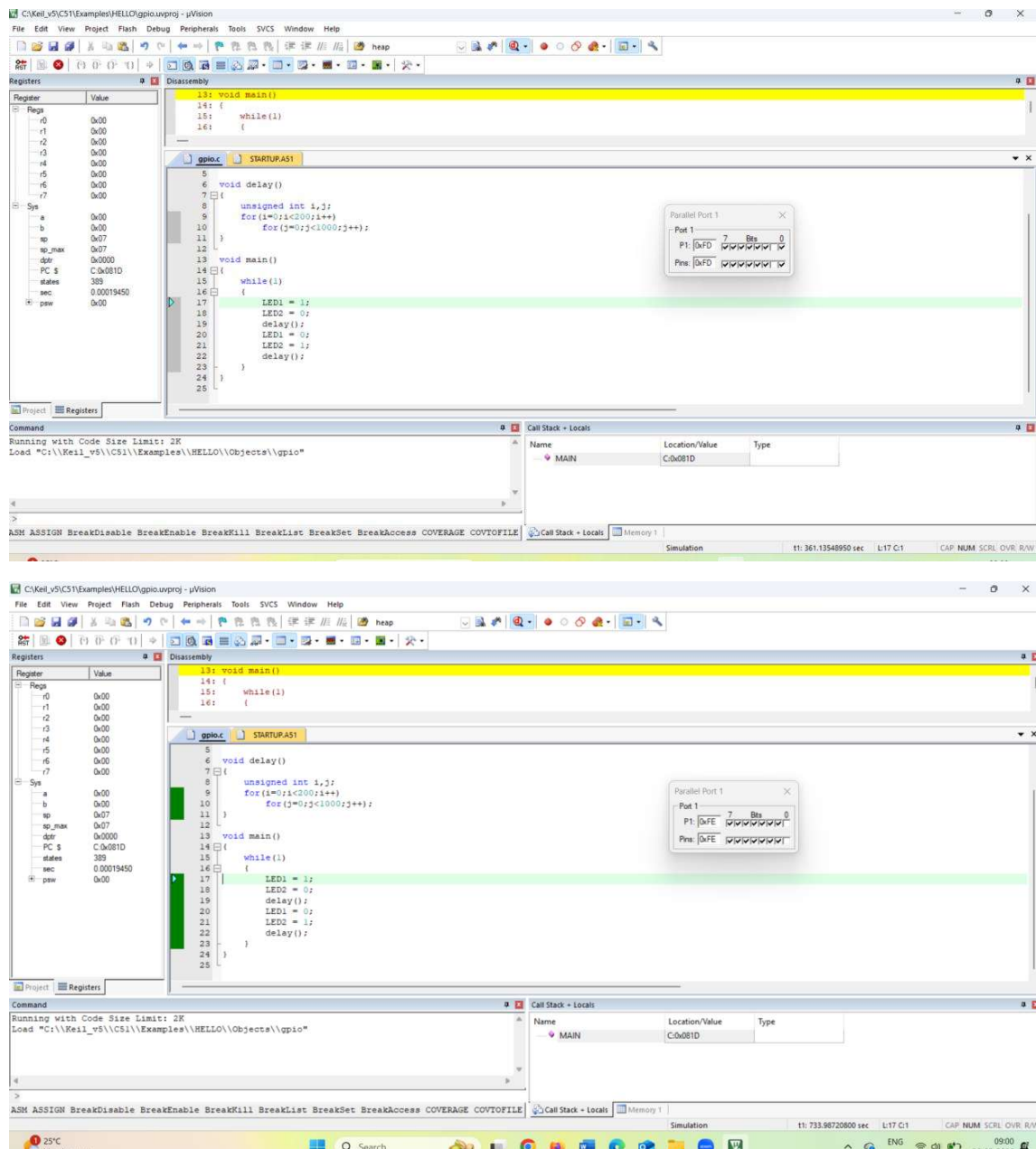
```
#include <reg51.h>

sbit LED1 = P1^0;
sbit LED2 = P1^1;

void delay()
{
    unsigned int i,j;
    for(i=0;i<200;i++)
        for(j=0;j<1000;j++);
}

void main()
{
    while(1)
    {
        LED1 = 1;
        LED2 = 0;
        delay();
        LED1 = 0;
        LED2 = 1;
        delay();
    }
}
```

OUTPUT:



OBERVATION:

Register	Value
P1	0x01
P1	0x02

RESULT:

Thus GPIO and bit addressing was studied and in Port1 the pins P1.0 and P1.1 toggled alternately.

Exp No: 12

EMBEDDED C PROGRAM TO MULTIPLY TWO 16-BIT BINARY NUMBERS

AIM:

To write an Embedded C program to multiply two 16-bit numbers using Keil software

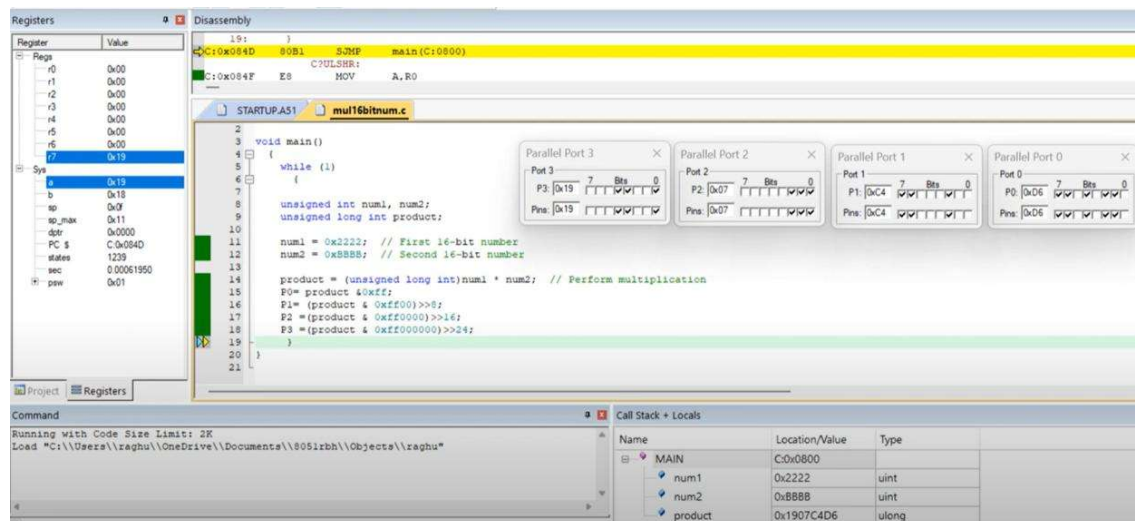
SOFTWARE REQUIRED:

- Keil

Program:

```
#include <reg51.h> void main()
{
while (1)
{
unsigned int num1, num2; unsigned long int product; num1 = 0x2222;
num2 = 0xB BBB;
product = (unsigned long int)num1 * num2; P0= product &0xff;
P1= (product & 0xff00)>>8;
P2 =(product & 0xff0000)>>16; P3 =(product & 0xff000000)>>24;
}
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 13

EMBEDDED C PROGRAM TO ADD AN ARRAY OF 16-BIT NUMBERS AND STORE THE 32-BIT RESULT IN INTERNAL RAM

AIM:

To write an Embedded C program to add an array of 16-bit numbers and store the results in internal RAM using Keil software.

SOFTWARE REQUIRED:

- Keil

PROGRAM:

```
#include <reg51.h>
```

```
void main(void)
```

```
{
```

```
    unsigned int i;
```

```
    array[00x00] unsigned int array[5] = {0x1111, 0x2222, 0x8888, 0x4444, 0xABCD};
```

```
    unsigned long sum = 0;
```

```
    for (i = 0; i < 5; i++)
```

```
    {
```

```
        sum = sum + array[i];
```

```
    }
```

```
    P0 = (unsigned char)(sum & 0xFF);    // Lower 8 bits
```

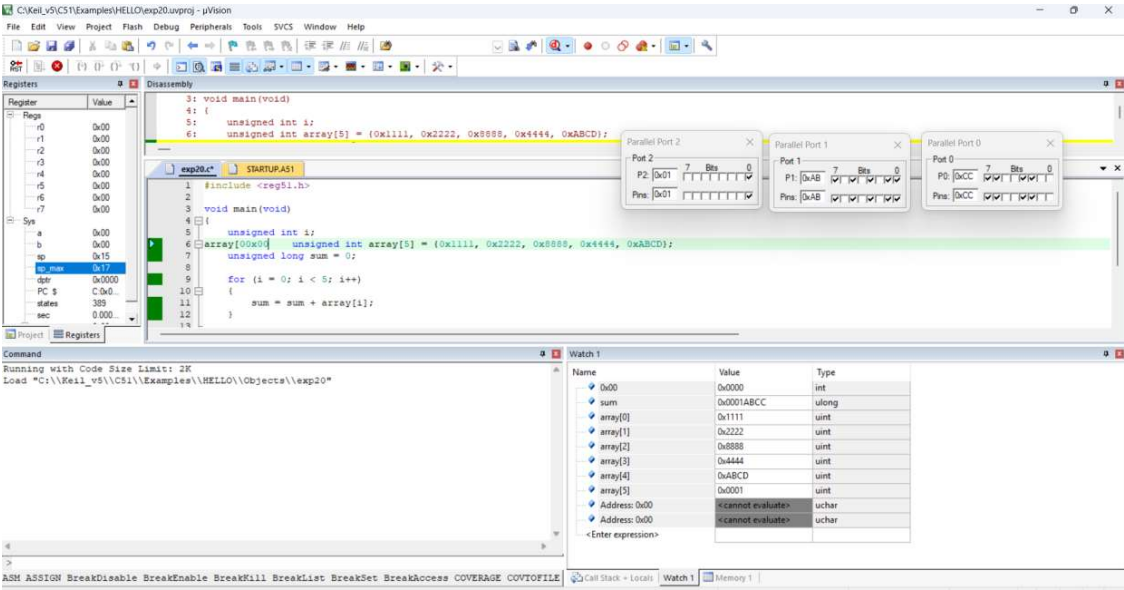
```
    P1 = (unsigned char)((sum >> 8) & 0xFF); // Next 8 bits
```

```
    P2 = (unsigned char)((sum >> 16) & 0xFF); // Upper 8 bits
```

```
    while(1); // keep output stable
```

```
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 14

EMBEDDED C PROGRAM TO DISPLAY “HELLO WORLD” MESSAGE

AIM:

To write an Embedded C program to display “HELLO WORLD” message using Keil software.

SOFTWARE REQUIRED:

- Keil software

Program:

```
#include <reg51.h> #include <stdio.h>
```

```
void main (void){
```

```
    SCON = 0x50;          /* SCON: mode 1, 8-bit UART, enable rcvr */ TMOD = 0x20;
                          /* TMOD: timer 1, mode 2, 8-bit reload */ TH1 = 0xFD;   /* TH1: reload
value for 9600 baudrate */
```

```
    TR1 = 1;             /* TR1: timer 1 run */
```

```
    TI = 1;              /* TI: set TI to send first char of UART */ while (1)
```

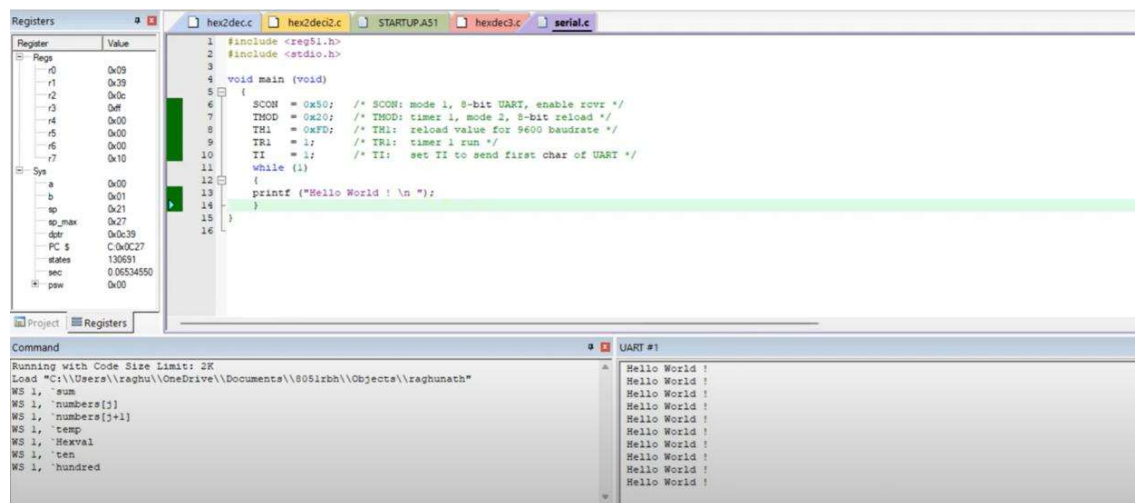
```
{
```

```
    printf ("Hello World ! \n ");
```

```
}
```

```
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.